

Design and Implementation of an Automated Wind-Tunnel Platform for Heatsink Thermal Characterisation

Alexander Groenvynck

Academic year 2025–2026

Bachelor in Energy Technology — Odisee University of Applied Sciences, Ghent

Exchange: University of Girona Supervisor: Lino Montoro Moreno

Table of Contents

1. The Problem
2. Research Questions
3. System Overview
4. Wind Tunnel & Hardware
5. Control Strategy
6. Software Architecture
7. The Automated Workflow
8. Results & Validation
9. Safety
10. Conclusion & Future Work

The Problem

Measuring heatsink performance by hand is:

- **Slow** — you must hold a set temperature and wait for equilibrium
- **Error-prone** — reading multiple instruments simultaneously
- **Unsafe** — a live heater is always energised
- **Cognitively expensive** — procedure overhead crowds out the learning objective

Goal: a system where the student presses **one button** and walks away with a result CSV.

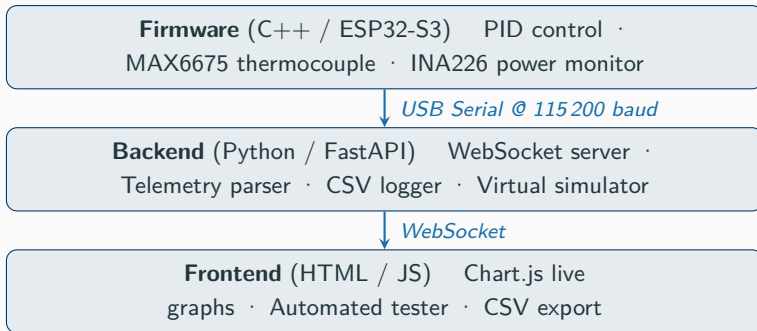
[PHOTO]

Complete assembled system
on the lab bench

Research Questions

- RQ1** Can a low-cost microcontroller hold a heat source at a series of set-point temperatures with sufficient **stability** for a repeatable measurement?
- RQ2** Can thermal **equilibrium be detected automatically**, removing the need for an operator to judge when the system has settled?
- RQ3** Can the complete test workflow be reduced to a **single student action** while guaranteeing safe shutdown of the heater under all termination conditions?
- RQ4** Does the platform produce a comparative thermal metric that **meaningfully distinguishes** heatsinks of differing geometry and material?

System Overview — Three-Tier Architecture



Wind Tunnel Construction

- Short rectangular enclosure — **chipboard panels**
- Fan at one end drives forced-convection airflow
- **Hexagonal honeycomb grid** immediately downstream of the fan
 - Breaks up rotational vortex flow from the fan blades
 - Produces a more **uniform, laminar-like velocity profile** across the test section
 - Critical for fair heatsink comparison: every sample sees the same airflow
- Heater module sits at the centre of the test section

[PHOTO]

Wind tunnel exterior
— full view

[PHOTO]

Fan end with HEX grid
close-up

The Heater Module

Stack (bottom to top):

1. Insulation material — prevents heat loss downward
2. **Steel backing plate** — structural support
3. **Ceramic PTC heater element** — the heat source
4. **Copper spreader plate** — distributes heat uniformly
5. Heatsink under test — clipped on with a wire retaining clip

Thermocouple: inserted into a precision-drilled hole in the copper plate — tracks heatsink base temperature directly.

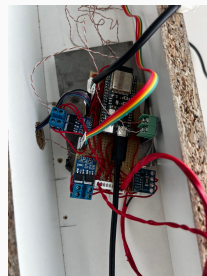
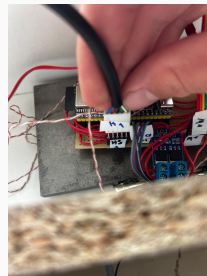
Built with: Miquel Lladó i Tuñà (copper plate, steel backing, insulation)



Key components:

- **ESP32-S3 DevKit-C1** — microcontroller, runs firmware
- **MAX6675** — K-type thermocouple-to-digital converter (SPI)
- **INA226** — bi-directional current & power monitor (I²C), shunt in series with heater → measures dissipated power directly
- **2× MOSFET modules** — switch heater (GPIO 4) and fan (GPIO 5)
- **Eventek KPS3010D** — 12 V / 10 A lab bench PSU

All electronics on a **perfboard dev board**, housed in the tunnel base.



Control Strategy: SMART Mode

Three firmware control modes: AUTO ·
MANUAL · SMART

SMART mode — used for all automated tests:

1. **PID drives** heater PWM toward set-point
2. Temperature filtered by **EMA** (exponential moving average)
+ glitch rejection + stuck-sensor watchdog
3. When stable within $\pm 0.5^\circ\text{C}$ for 8 consecutive seconds:
enter **HOLD** — lock equilibrium PWM
4. Backend independently checks **EqPWM convergence**
(< 0.5 PWM units between frames) before recording

Why lock the PWM?

At steady state, the equilibrium PWM directly tells us how much power the heater is delivering. Combined with the INA226 current reading, this gives us the dissipated power P — which feeds directly into $R_{th} = \Delta T / P$.

Why dual-check?

Firmware stability band catches noise in the sensor. Backend EqPWM convergence catches slow drift that passes the firmware test but isn't truly settled.

Thermal Metric: Thermal Resistance R_{th}

$$R_{th} = \frac{\Delta T}{P} \quad \left[\frac{^{\circ}\text{C}}{\text{W}} \right]$$

where $\Delta T = T_{heatsink} - T_{ambient}$

and P = electrical power dissipated by the heater [W]

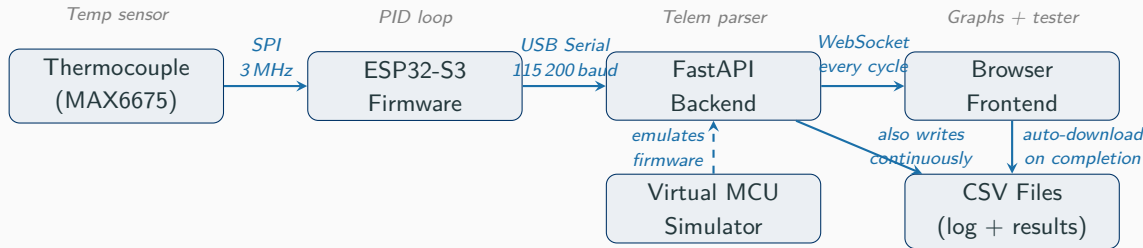
- **Lower** R_{th} = heat removed more effectively = **better heatsink**
- Measured at multiple set-points — R_{th} should be **constant** (material/geometry property)
- Comparison is **fair** because airflow is identical for every heatsink tested

Intuition:

A heatsink with **fins** has far more convective surface area than a plain block, even if the plain block is made of copper (higher conductivity). Under forced airflow, surface area dominates — so a finned aluminium heatsink typically outperforms a solid copper block.

This is exactly what the results show.

Software Architecture: Data Flow



Backend: `server.py`

Key responsibilities:

- **`_serial_reader()`** — async loop, reads one line at a time from the COM port at 115,200 baud
- **`_handle_line()`** — applies `TELEM_RE` (single regex) to extract all telemetry fields; calls `broadcast()`
- **`broadcast()`** — pushes a JSON "telemetry" message to every connected WebSocket client
- **`sendCmd()`** — sends SET / GET commands to the firmware
- **CSV logger** — appends each frame to the raw log; `CSV_FIELDS` + `schema_version`

Virtual MCU Simulator:

Selecting the VIRTUAL port runs a software emulator that produces realistic telemetry — PID behaviour, equilibrium detection, ramp-up — with no hardware.

Used to develop and debug the entire tester workflow before the hardware existed.

WebSocket messages (type field):

"telemetry" · "ports" ·
"connected"
"disconnected" · "error"

Frontend: index.html

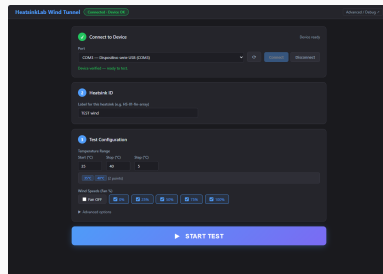
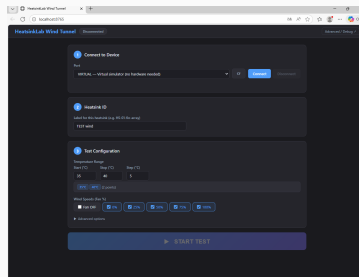
Single file — all HTML, CSS, and JavaScript
(No build step, no Node.js, no internet required)

WebSocket onmessage handler:

- Receives "telemetry" every firmware cycle
- Updates **Chart.js** live temperature trace
- Updates sidebar readouts (temp, power, PWM, fan speed...)

Tester state machine (JavaScript):

- Steps through the programmed set-points
- Watches `eq_pwm` convergence on each telemetry frame
- When converged: records a result row
- Advances to next set-point — or ends test

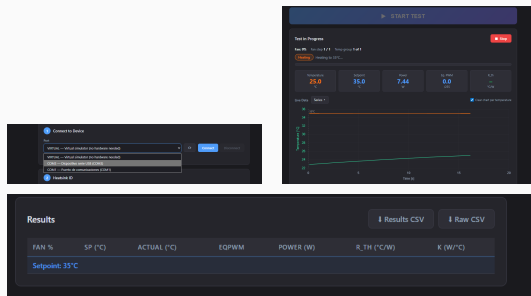


The Automated Workflow

Student steps (all of them):

1. Plug in ESP32 via USB
2. Double-click *HEATSINK TESTER* shortcut
3. Select COM port → click Connect
4. Type heatsink ID (e.g. HS-01-fin-array)
5. Press **Start Test**
6. **Wait** — system steps through all temperatures automatically
7. Results CSV downloads automatically when complete
8. Swap heatsink → repeat from step 4

Students never need to know what PID is.



Results: R_{th} Comparison

Heatsink		SP (°C)	P (W)	R_{th} (°C/W)
HS-01 fin-array		40	7.6	2.38
HS-01 fin-array		50	11.6	2.42
HS-01 fin-array		60	15.9	2.38
HS-02	copper block	40	4.4	4.05
HS-02	copper block	50	7.0	3.99
HS-02	copper block	60	9.5	3.97

Representative values — illustrate output format.

Key observations:

- R_{th} is **consistent across set-points** for each heatsink — confirms steady-state measurement quality
- **Finned aluminium (HS-01) outperforms solid copper (HS-02)** despite copper's higher conductivity: fins provide far more convective surface area
- HS-01 needs **more power** to hold the same temperature — more heat is being removed by the airflow, so the heater has to work harder
- Platform **clearly distinguishes** the two samples: 2.4 vs 4.0 °C/W

Safety: Heater Shutdown

Three tested termination paths — all verified:

1. **Normal test completion**

Backend sends SET RUN OFF automatically after the last set-point. GUI displays *"Test complete — heater off"*.

2. **Manual Stop button**

Immediately sends SET RUN OFF. Heater power drops to zero within one firmware cycle.

3. **Browser close / network disconnect**

Backend detects WebSocket disconnection and sends SET RUN OFF as a fallback.

In all three cases: INA226 power reading returned to zero.

Why this matters:

A heater left on unattended in a lab is a genuine fire and burn risk. Requiring a student to manually cut power after the test means relying on a human action that can be forgotten.

Automatic shutdown on every exit path removes that dependency entirely.

Additionally: a 60-second cool-down at low fan speed runs before the fan stops, reducing thermal stress on components.

Limitations & Future Work

Current limitations:

- **Airspeed commanded, not measured** — fan % assumes reproducible fan curve; real m/s unknown
- **No independent hardware safety watchdog** — safety relies on firmware + backend logic; a separate MCU with its own power cut would be more robust
- **Single thermocouple** — no independent ambient sensor; T_{amb} estimated manually
- **No formal repeatability study** — functional validation only; uncertainty budget not quantified

Highest-priority next steps:

1. Wire the **anemometer** and **SDP510 pressure sensor** — already provisioned in the telemetry schema and CSV; enables closed-loop airspeed control
2. Add an **independent hardware watchdog MCU** with a power-cut relay
3. Run a structured **repeatability study** on reference heatsinks
4. Record **ambient temperature and humidity** per run (EZO-HUM already wired)
5. Packaged **one-click installer** for lab deployment

Conclusion

All four research questions answered:

RQ1 PID + EMA: stable within $\pm 0.5^\circ\text{C}$

RQ2 Dual-check equilibrium detection: reliable

RQ3 One button; shutdown on all exit paths

RQ4 Platform distinguishes heatsinks clearly

Key contributions:

- Robust, safe, genuinely usable lab automation system
- **Software-first architecture:** firmware locked, all features in backend + frontend — rapid iteration, hardware unchanged from first prototype to final build
- Any engineering student can characterise a heatsink *without knowing what PID is*

[PHOTO]

Complete assembled system
on the lab bench

Thank you.

Questions?